# The role of irrelevant variables in linear models

This notebook demonstrates overfitting and importance to remove irrelevant variables from a statistical model.

## Artificial data

We will generate artificial data first. The dataset contains 100 independent variables and one dependent variable, all continuous. All the variables follow the normal distribution with zero mean and unit variance. Notice that there is **no relationship between dependent and independent variables**. Consequently, all further findings must be false positives (Type I error).

```
set.seed(5) # you can change the seed or remove the line to see more runs
d<-data.frame(matrix(rnorm(200*100,0,1),nrow=200))
d$out<-rnorm(200,0,1)
```

## Learn a linear model

Next, we will create a linear regression model. There is no feature selection in this step. Check the number of seemingly relevant independent variables, their number should be around alpha*nvars=5. You may need to generate data multiple times to get an "illustrative" case.

```
lm.all<-lm(out ~ .,data=d)
lm.all.par<-coefficients(summary(lm.all))
sum(lm.all.par[,"Pr(>|t|)"]<0.05)
```

```
## [1] 1
```

```
summary(lm.all) # print out the number of significant independent variables
```

```
##
## Call:
## lm(formula = out ~ ., data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.33639 -0.51293  0.02175  0.50097  1.65287
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.0541930  0.1158497   0.468   0.6410
## X1          -0.0437721  0.1145602  -0.382   0.7032
## X2          -0.0113185  0.1187796  -0.095   0.9243
## X3          -0.0515659  0.1124812  -0.458   0.6476
## X4           0.1290264  0.1048162   1.231   0.2212
## X5           0.0175343  0.1136667   0.154   0.8777
## X6          -0.0802444  0.1141263  -0.703   0.4836
## X7          -0.1212466  0.1112893  -1.089   0.2786
## X8          -0.0154321  0.1108775  -0.139   0.8896
## X9          -0.0014912  0.1068265  -0.014   0.9889
```

```
## X10         -0.0194076  0.1144602  -0.170   0.8657
## X11         -0.0369361  0.1066724  -0.346   0.7299
## X12         -0.0441677  0.1272358  -0.347   0.7292
## X13         -0.0326989  0.1038578  -0.315   0.7535
## X14         -0.0764064  0.1114877  -0.685   0.4947
## X15         -0.1119700  0.1131989  -0.989   0.3250
## X16          0.0274618  0.1174395   0.234   0.8156
## X17         -0.2293726  0.1128082  -2.033   0.0447 *
## X18         -0.1568718  0.1126065  -1.393   0.1667
## X19          0.1338106  0.1162933   1.151   0.2527
## X20         -0.0041374  0.1041486  -0.040   0.9684
## X21         -0.0314420  0.1103852  -0.285   0.7764
## X22          0.0005378  0.1098945   0.005   0.9961
## X23          0.1396243  0.1185245   1.178   0.2416
## X24         -0.0654729  0.1157383  -0.566   0.5729
## X25         -0.0490701  0.1133353  -0.433   0.6660
## X26         -0.0900963  0.1195647  -0.754   0.4529
## X27          0.1356819  0.1027708   1.320   0.1898
## X28          0.0921824  0.1121639   0.822   0.4131
## X29          0.0612537  0.1161212   0.527   0.5990
## X30          0.1575238  0.1062987   1.482   0.1415
## X31          0.0982322  0.1052015   0.934   0.3527
## X32          0.0540137  0.1128386   0.479   0.6332
## X33          0.0200546  0.1108073   0.181   0.8567
## X34         -0.0342363  0.1229323  -0.278   0.7812
## X35          0.1246611  0.1071795   1.163   0.2476
## X36          0.1553035  0.0910022   1.707   0.0910 .
## X37          0.0930225  0.1035502   0.898   0.3712
## X38          0.0019701  0.1028252   0.019   0.9848
## X39         -0.0241003  0.1208368  -0.199   0.8423
## X40         -0.1178047  0.1179508  -0.999   0.3203
## X41          0.0129882  0.1116658   0.116   0.9076
## X42         -0.0616584  0.1122973  -0.549   0.5842
## X43         -0.0359595  0.1081310  -0.333   0.7402
## X44          0.0578856  0.1113412   0.520   0.6043
## X45          0.0558542  0.1249363   0.447   0.6558
## X46          0.0740797  0.1002976   0.739   0.4619
## X47          0.0637614  0.0977710   0.652   0.5158
## X48          0.0163021  0.1081793   0.151   0.8805
## X49         -0.0132357  0.1113670  -0.119   0.9056
## X50         -0.1519989  0.1133706  -1.341   0.1831
## X51          0.1528841  0.1193138   1.281   0.2031
## X52         -0.0168458  0.1081409  -0.156   0.8765
## X53         -0.1537687  0.1170663  -1.314   0.1920
## X54         -0.0084867  0.1109728  -0.076   0.9392
## X55         -0.0002220  0.1201929  -0.002   0.9985
## X56          0.1323080  0.1129297   1.172   0.2442
## X57          0.0328376  0.1108739   0.296   0.7677
## X58         -0.0382010  0.1196205  -0.319   0.7501
## X59         -0.0850259  0.1062094  -0.801   0.4253
## X60          0.0488110  0.1192611   0.409   0.6832
## X61          0.0020084  0.1135041   0.018   0.9859
## X62          0.0120774  0.1079625   0.112   0.9112
## X63         -0.0475840  0.1163142  -0.409   0.6834
```

```
## X64          0.1059578  0.1070238   0.990   0.3246
## X65          0.1194900  0.1284213   0.930   0.3544
## X66          0.0219762  0.1138911   0.193   0.8474
## X67          0.0614538  0.1191804   0.516   0.6073
## X68         -0.0203136  0.1176076  -0.173   0.8632
## X69          0.0556333  0.1096509   0.507   0.6130
## X70          0.0236512  0.1137815   0.208   0.8358
## X71         -0.0839413  0.1113258  -0.754   0.4526
## X72         -0.0202959  0.1210615  -0.168   0.8672
## X73         -0.0666444  0.1122227  -0.594   0.5540
## X74          0.2048668  0.1344193   1.524   0.1307
## X75         -0.0236298  0.1046323  -0.226   0.8218
## X76          0.0238431  0.1040374   0.229   0.8192
## X77         -0.0013974  0.1279640  -0.011   0.9913
## X78          0.0163050  0.1113074   0.146   0.8838
## X79         -0.0148050  0.1176053  -0.126   0.9001
## X80          0.0075310  0.1279998   0.059   0.9532
## X81          0.0449086  0.1139525   0.394   0.6944
## X82         -0.1189056  0.1065121  -1.116   0.2670
## X83          0.0576454  0.0973877   0.592   0.5553
## X84          0.1818205  0.1133132   1.605   0.1118
## X85          0.0230971  0.1116037   0.207   0.8365
## X86          0.0506234  0.1088342   0.465   0.6429
## X87          0.0255942  0.1161022   0.220   0.8260
## X88          0.1247000  0.1097515   1.136   0.2586
## X89          0.0465110  0.1051696   0.442   0.6593
## X90         -0.0960533  0.1073296  -0.895   0.3730
## X91         -0.0524689  0.1167084  -0.450   0.6540
## X92         -0.0724895  0.0992926  -0.730   0.4671
## X93          0.0140036  0.1162881   0.120   0.9044
## X94          0.1631527  0.1236142   1.320   0.1899
## X95         -0.1120812  0.1135115  -0.987   0.3259
## X96          0.1095906  0.1142111   0.960   0.3396
## X97         -0.0730748  0.1227972  -0.595   0.5531
## X98         -0.0570898  0.1140662  -0.500   0.6178
## X99          0.1103782  0.1071685   1.030   0.3055
## X100         0.0313958  0.1026592   0.306   0.7604
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.124 on 99 degrees of freedom
## Multiple R-squared:  0.4462, Adjusted R-squared:  -0.1132
## F-statistic: 0.7977 on 100 and 99 DF,  p-value: 0.8695
```

There were 4 independent variables looking significant. On the other hand, both Adjusted R-squared and F-statistic suggest that the full model should not be used and none of its independent variables is truly relevant.

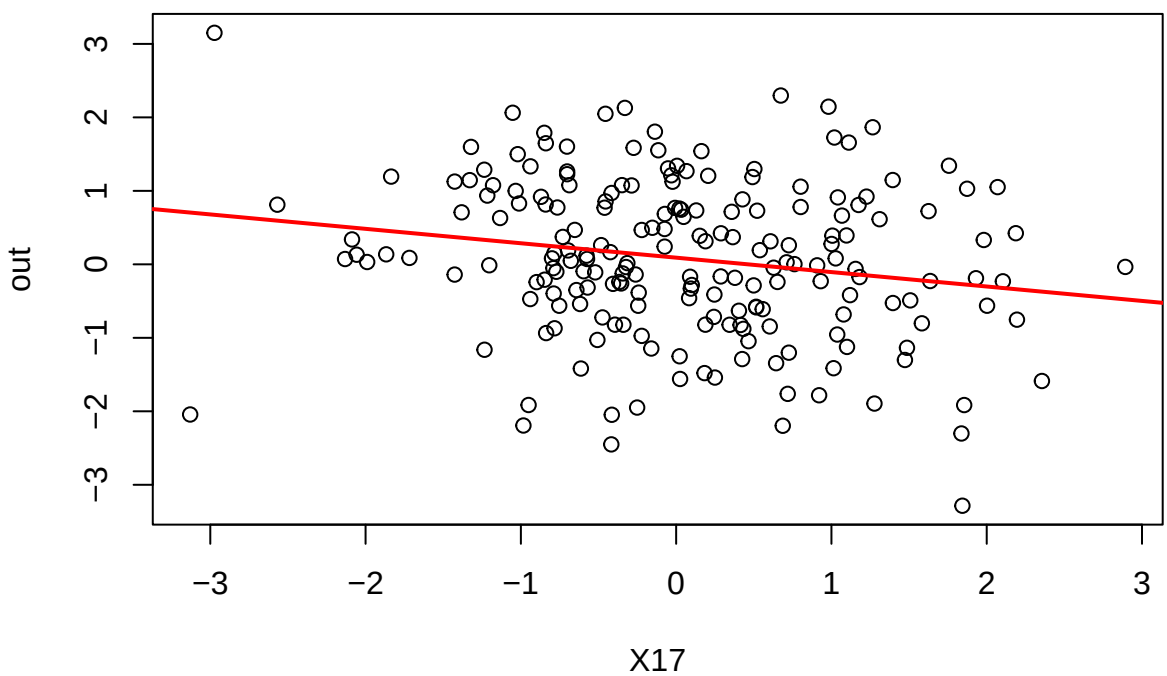
## See the best predictor

Let us pick the best predictor/independent variable and visualise its relationship with the dependent variable. Both correlation and visual fit look significant, still, it is a false positive due to **selection bias**. The significance is only the outcome of selection of the best variable out of a large pool of irrelevant variables.

```r
pred<-rownames(lm.all.par)[which(lm.all.par[,"Pr(>|t|)"]==min(lm.all.par[,"Pr(>|t|)"]))] # find the mos
cor.test(d[[pred]],d$out)
```

```
##
##  Pearson's product-moment correlation
##
## data:  d[[pred]] and d$out
## t = -2.7571, df = 198, p-value = 0.006377
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  -0.32242168 -0.05500851
## sample estimates:
##       cor
## -0.192282
```

```r
plot(d[[pred]], d$out,xlab=pred,ylab="out") # visualize the relationship between indep and dep variable
abline(lm(as.formula(paste("out~",pred)), d),col="red", lw=2) # visualize the fit
```




The pairwise correlation between the best predictor and the outcome is significant. Note that the correlation coefficient and its p-value may differ from the coefficient and p-value of the given variable observed in the multivariate model above (paired vs multivariate analysis).

## Alternative regression models

There are four alternative linear models to consider: 1. lm.all ... the full model, use all 100 independent variables, 2. lm.sel ... use only the (seemingly) relevant variables, 3. lm.simple ... use only the intercept, 4. lm.zero ... always predict zero.

Knowing the origin of data, it is obvious that it holds lm.all<lm.sel<lm.simple<lm.zero when considering model performance on unseen data. The best prediction is to always predict zero. The other linear models will be constructed as follows:

```r
sel.form<-as.formula(paste("out~",paste(rownames(lm.all.par)[lm.all.par[,"Pr(>|t|)"]<0.05],collapse="+"
lm.sel<- lm(sel.form, data=d) # obviously, it is better to employ lasso or stepwise regression, this is
```

```
lm.simple<-lm(out ~ 1,data=d)
lm.zero<-lm(out ~ 0,data=d)
```

## Compare the alternative models, find the best one

### Large samples, holdout method

Let us suppose the common situation of not knowing the true relationships among variables. The goal is to find the best model, i.e., the model that represents a trade-off between sufficient flexibility and minimal overfitting. In this case, we deal with artificial data and we can easily generate previously unseen test samples. **The holdout method** makes the simplest testing choice here:

```
d_test<-data.frame(matrix(rnorm(1000*100,0,1),nrow=1000))
d_test$out<-rnorm(1000,0,1)

myRMSE(predict(lm.all,d_test),d_test$out)
```

```
## [1] 1.317244
```

```
myRMSE(predict(lm.sel,d_test),d_test$out)
```

```
## [1] 1.009906
```

```
myRMSE(predict(lm.simple,d_test),d_test$out)
```

```
## [1] 0.992419
```

```
myRMSE(predict(lm.zero,d_test),d_test$out)
```

```
## [1] 0.9845113
```

The zero model shows the smallest root mean squared error. At the same time, the model has the smallest variance in this error (not shown). The holdout method ranks the linear models correctly. The absolute value of calculated RMSE for the zero model matches with the amount of irreducible error (the value of 1 as we deal with the outcome that has zero mean and unit variance).

### Small samples, ANOVA and cross-validation

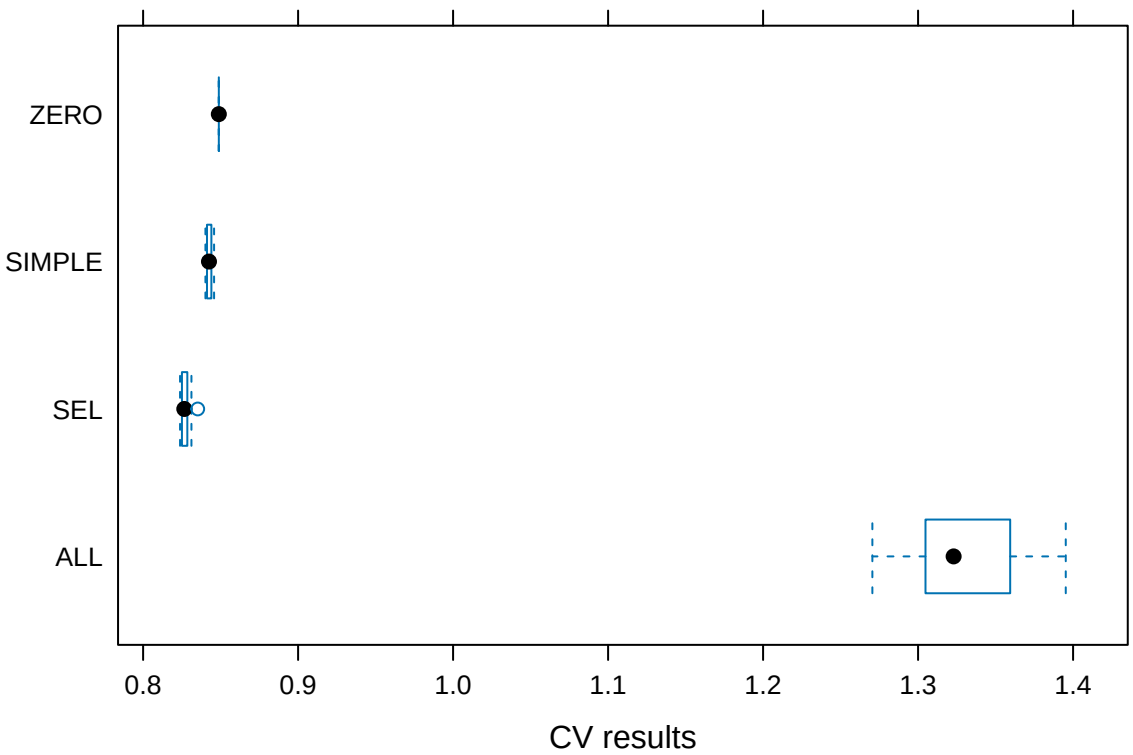
Often, the sample set is small and new observations cannot be easily reached. Under such circumstances, ANOVA and cross-validation are available to compare the models. Let us start with ANOVA:

```
anova(lm.zero,lm.simple,lm.sel,lm.all)
```

```
anova(lm.zero,lm.all)
```

ANOVA may often recommend lm.sel, which is not correct. The method may suffer from conceptual simplicity of the test (no need for test data, the decision made on train data only). However, ANOVA does not prefer lm.all over lm.zero. In general, it is a legitimate option. Let us employ cross-validation now:

```
folds <- cvFolds(nrow(d), K = 10, R = 10) # 10 times repeated 10-fold CV
cvFitLmAll <- cvLm(lm.all, cost = rtmspe, folds = folds, trim = 0.1)
cvFitLmSel <- cvLm(lm.sel, cost = rtmspe, folds = folds, trim = 0.1)
cvFitSimple <- cvLm(lm.simple, cost = rtmspe, folds = folds, trim = 0.1)
cvFitZero <- cvLm(lm.zero, cost = rtmspe, folds = folds, trim = 0.1)
cvFits <- cvSelect(ALL = cvFitLmAll, SEL = cvFitLmSel, SIMPLE = cvFitSimple, ZERO=cvFitZero)
bwplot(cvFits) # rmse is the default cost function
```




The recommendation is misleading again, sel is best, full model worst. The problem was methodological in this case, **we cannot do feature selection on all the data**, we must proceed independently in all the folds! We will not implement the correct way of cross-validation here. However, remember that correct CV implementation could be non-trivial when doing learning, data preprocessing and hyperparameter tuning at the same time.

## Summary

Irrelevant features cause overfitting and make our models work worse on unseen data. If having a finite/limited sample set the learning algorithm finds spurious relationships which increases variance and thus error. Removal of irrelevant features is crucial namely when dealing with a large number of them. (Proper) testing on unseen data (hold-out method, cross-validation) can help to detect overfitting and find out the optimal complexity of the model. ANOVA helps to decide the complexity from one run of the model only.

**Further questions and tasks:**

1. Show how the previously learned feature selection methods (p-values, stepwise selection, shrinkage) work in this case. Clearly demonstrate whether they work well/fail and explain why. Play with several different random generator seeds.
2. Describe step by step the correct way of model comparison procedure through cross-validation that was incorrectly implemented above. You can also implement the procedure, however, the implementation is optional only.